Live Demo: Tokenize, Postfix, Evaluate

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 19 Tuesday, November 10, 2026

The Pipeline We Built on Nov 5

```
"12 + 3 * (1 - 2) / 4"
        ↓ tokenize
queue<string>: 12 + 3 * ( 1 - 2 ) / 4
        ↓ toPostfix
queue<string>: 12 3 1 2 - * 4 / +
        ↓ evalPostfix
double: 11.25
```

Think of an assembly line: each function does one job and hands its result to the next. The **types must match** at every arrow.

Demo 1: tokenize

```
std::queue<std::string> tokenize(const std::string& s) {
  std::queue<std::string> tokens;

  int len = static_cast<int>(s.length());
  int i = 0;
  while (i < len) {
    if (s[i] == ' ') {
      i++;
    }
    else if (std::isdigit(static_cast<unsigned char>(s[i]))) {
      std::string numStr;
      while (i < len && std::isdigit(static_cast<unsigned char>(s[i]))) {
        numStr += s[i];
        i++;
      }
      tokens.push(numStr);
    }
    else if (s[i] == '(' || s[i] == ')' ||
             s[i] == '+' || s[i] == '-' ||
             s[i] == '*' || s[i] == '/') {
      tokens.push(std::string(1, s[i]));
      i++;
    }
    else {
      // Without this branch, an unsupported character would cause
      // an infinite loop because i would not advance.
      throw std::invalid_argument("Invalid character in infix expression");
    }
  }

  return tokens;
}
```

Tokenizer: Where to Look

- Skip a space; `i` advances by one.
- A digit starts a number: keep reading **while** the next character is a digit, testing `i < text.size()` **first**.
- One of `( ) + - * /` is a token by itself.
- Anything else: `throw`. Otherwise `i` never advances and the loop never ends.

Demo 2: `prec` and the "(" Trick

```
int prec(const std::string& op) {
  if (op == "(") {
    return 0;
  }
  else if (op == "+" || op == "-") {
    return 1;
  }
  else if (op == "*" || op == "/") {
    return 2;
  }

  return -1;
}
```

Demo 3a: `toPostfix`: Numbers and Operators

```
// shunting yard algorithm
std::queue<std::string> toPostfix(std::queue<std::string> infix) {
  std::queue<std::string> postfix;
  std::stack<std::string> operators; // waiting to be added to postfix
  while (!infix.empty()) {
    std::string token = infix.front();
    infix.pop();
    if (std::isdigit(static_cast<unsigned char>(token[0]))) {
      postfix.push(token);
    }
    else if (token == "+" || token == "-" ||
             token == "*" || token == "/") {
      // Pop operators of higher or equal precedence.
      // Stop at ( or an operator of lower precedence.
      while (!operators.empty() && prec(operators.top()) >= prec(token)) {
        postfix.push(operators.top());
        operators.pop();
      }

      // After the loop, either:
      // 1. operators.empty(), or
      // 2. !operators.empty() &&
      //    prec(operators.top()) < prec(token).
      // In case 2, the top is '(' or an operator of lower precedence.
      operators.push(token);
    }
```

Demo 3b: `toPostfix`: Parentheses and Cleanup

```
    else if (token == "(") {
      operators.push(token);
    }
    else if (token == ")") {
      while (!operators.empty() && operators.top() != "(") {
        postfix.push(operators.top());
        operators.pop();
      }

      // After the loop, if the operator stack is not empty,
      // its top element must be '('.
      if (operators.empty()) {
        throw std::invalid_argument("Unmatched closing parenthesis");
      }
      operators.pop(); // pop the matched '('
    }
    else {
      throw std::invalid_argument("Invalid token in infix expression");
    }
  }

  while (!operators.empty()) {
    // An unmatched ( must not be added to the postfix expression.
    if (operators.top() == "(") {
      throw std::invalid_argument("Unmatched opening parenthesis");
    }
    postfix.push(operators.top());
    operators.pop();
  }

  return postfix;
}
```

Shunting Yard: Three Rules to Say Out Loud

1. **Number** $\Rightarrow$ straight to the output.
2. **Operator** $\Rightarrow$ pop everything of **higher or equal** precedence first, then push it. "Equal" gives left-to-right: `8-3-2` is `8 3 - 2 -`.
3. **Parentheses**: push `(`. On `)`, pop until `(`, then discard it.

Because `prec("(")` is 0, no operator ever pops a `(`. Only `)` can.

Demo 4a: `evalPostfix`: Reading Tokens

```
double evalPostfix(std::queue<std::string> postfix) {
  std::stack<double> operands;
  while (!postfix.empty()) {
    std::string token = postfix.front();
    postfix.pop();
    char ch = token[0];
    if (std::isdigit(static_cast<unsigned char>(ch))) {
      operands.push(std::stod(token));
    }
    else if (ch == '+' || ch == '-' || ch == '*' || ch == '/') {
      if (operands.size() < 2) {
        throw std::invalid_argument("need at least two operands");
      }

      double right = operands.top();
      operands.pop();
      double left = operands.top();
      operands.pop();
```

Demo 4b: `evalPostfix`: Applying an Operator

```
      switch (ch) {
        case '+': operands.push(left + right);
                  break;
        case '-': operands.push(left - right);
                  break;
        case '*': operands.push(left * right);
                  break;
        case '/': if (right == 0.0) {
                    throw std::invalid_argument("division by zero");
                  }
                  operands.push(left / right);
                  break;
      }
    }
    else {
      throw std::invalid_argument("Invalid token in postfix expression");
    }
  }

  // A valid postfix expression leaves exactly one value on the stack.
  if (operands.size() != 1) {
    throw std::invalid_argument("invalid expression");
  }

  return operands.top();
}
```

Trace the Evaluation: `12 3 1 2 - * 4 / +`

token read	operand stack (top on the right)	
`12 3 1 2`	`12 3 1 2`	
`-`	`12 3 -1`	$(1-2)$
`*`	`12 -3`	$(3\cdot(-1))$
`4`	`12 -3 4`	
`/`	`12 -0.75`	$(-3/4)$
`+`	`11.25`	$(12+(-0.75))$

The **right** operand comes off the stack first. Swap the two pops and `-` and `/` give wrong answers.

Try These During the Demo

Input	Postfix	Result
`2 + 3 * 4`	`2 3 4 * +`	14
`(2 + 3) * 4`	`2 3 + 4 *`	20
`8 - 3 - 2`	`8 3 - 2 -`	3
`100 / 8 / 5`	`100 8 / 5 /`	2.5
`8 / (4 - 4)`	`8 4 4 - /`	error: division by zero
`(1 + 2`	—	error: unmatched (
`3 $ 4`	—	error: invalid token

Predict first, then run. Which of these would break if >= in `toPostfix` became >?